1. What is a Server?

A server is a computer or software that receives requests, processes them, and sends responses to other computers or applications.

সহজভাবে: Server হলো এমন একটি system, যেটি request গ্রহণ করে এবং সেই অনুযায়ী response দেয়।

Example: তুমি একটি website-এ login করলে, server তোমার login information যাচাই করতে পারে।

Client (Browser) - Request পাঠায়

Server - Request process করে

Database - Data সংরক্ষণ করে

সাধারণ flow: Client ↔ Server ↔ Database

- Client: যে application request পাঠায়, যেমন browser।
- Server: Request অনুযায়ী কাজ করে এবং response দেয়।
- Database: যেখানে data সংরক্ষণ করা হয়।

মনে রাখবে: Server ও database এক জিনিস নয়। Server database থেকে data নিয়ে client-কে দিতে পারে।

2. What is an API?

API (Application Programming Interface) is a defined way for two software systems to communicate with each other.

সহজভাবে: API হলো এমন একটি interface বা নিয়ম, যার মাধ্যমে একটি application অন্য application বা server-এর সঙ্গে যোগাযোগ করে।

API endpoint এমনভাবে তৈরি করা হয়, যাতে নির্দিষ্ট request পাঠালে server নির্দিষ্ট format-এ response দেয় — সাধারণত JSON।

API endpoint: কোথায় request পাঠাবে।

Important API terms

- Endpoint: API-এর নির্দিষ্ট address, যেখানে request পাঠানো হয়। যেমন `/api/products`।
- `https://example.com/api/products` → API endpoint-এর URL।
- Request: তুমি `fetch()` দিয়ে API endpoint-এ request পাঠাচ্ছ।
- Server request গ্রহণ করে।
- Server প্রয়োজন অনুযায়ী database থেকে data নেয়।
- Server JSON response পাঠায়।
- JSON: API-তে data আদান-প্রদানের একটি প্রচলিত format।

একটা সহজ উদাহরণ দিই।

ধরো, একটা restaurant-এ খাবার অর্ডার করছ:

- API → অর্ডার দেওয়ার নিয়ম: কীভাবে অর্ডার করবে, কী তথ্য দিতে হবে, কী response পাবে।
- API endpoint → অর্ডার দেওয়ার নির্দিষ্ট address বা জায়গা।
- Server → অর্ডার গ্রহণ করে প্রয়োজনীয় কাজ করে।
- Database → খাবার, দাম বা অর্ডারের তথ্য সংরক্ষণের জায়গা।
- Response → তোমার অর্ডারের ফলাফল।

এই analogy-তে endpoint-কে শুধু URL হিসেবে ভাবতে পারো, কিন্তু API বলতে পুরো communication interface-টিকে বোঝায়।

মনে রাখবে: API হলো communication interface; server হলো request process করে response দেওয়ার system। API নিজে server বা database নয়।

3. What is CRUD?

CRUD stands for Create, Read, Update, and Delete. এগুলো database-এর data নিয়ে কাজ করার চারটি মৌলিক operation।

C — Create

নতুন data তৈরি বা সংরক্ষণ করা।

Example: নতুন product যোগ করা।

HTTP method: `POST`

R — Read

আগে থেকে থাকা data দেখা বা সংগ্রহ করা।

Example: সব product-এর তালিকা দেখা।

HTTP method: `GET`

U — Update

আগে থেকে থাকা data পরিবর্তন করা।

Example: product-এর দাম পরিবর্তন করা।

HTTP method: `PUT` বা `PATCH`

D — Delete

কোনো data মুছে ফেলা।

Example: একটি product সরিয়ে ফেলা।

HTTP method: `DELETE`

4. What is JavaScript?

JavaScript is a programming language used to make websites interactive.

For example, JavaScript can:

- Handle button clicks.
- Validate form inputs.
- Update website content.
- Fetch data from an API.

Traditionally, JavaScript runs in a web browser using a JavaScript engine.

5. What is Node.js?

Node.js is a JavaScript runtime environment that allows us to run JavaScript outside the browser.

সহজভাবে, JavaScript হলো ভাষা, আর Node.js হলো সেই ভাষা চালানোর একটি environment, যা browser-এর বাইরেও JavaScript চালাতে পারে।

Browser + JavaScript engine

JavaScript চালিয়ে website interactive করে।

Example: Button click, form validation, DOM manipulation.

Node.js + JavaScript engine

Browser ছাড়াই JavaScript চালাতে পারে।

Example: Server তৈরি, file read/write, API তৈরি।

6. Why do we need Node.js when we already have JavaScript?

The key point is: JavaScript is the language; Node.js provides a runtime and additional capabilities.

ধরো, তুমি একটি website তৈরি করছ, যেখানে user login করবে এবং product-এর data database-এ save হবে।

শুধু browser-এর JavaScript দিয়ে পুরো backend তৈরি করা সাধারণত সম্ভব নয়, কারণ browser-এর JavaScript-এর সরাসরি server-side capabilities নেই।

Node.js ব্যবহার করে তুমি JavaScript দিয়েই backend তৈরি করতে পারবে।

Task	Browser JavaScript	Node.js
Button click handle	Yes	Not its primary purpose
Website UI update	Yes	Not directly
Backend server তৈরি	Not normally	Yes
Server-side file access	Restricted	Yes, with Node.js APIs and permissions
API তৈরি	Not normally	Yes
Database-এর সঙ্গে backend থেকে কাজ	Not normally	Yes, using suitable libraries/drivers

7. What is a Database?

A database is an organized collection of data that we can store, retrieve, update, and delete.

সহজভাবে, Database হলো এমন একটি জায়গা যেখানে application-এর data গুছিয়ে রাখা হয়, যাতে প্রয়োজন হলে সেই data ব্যবহার করা যায়।

Example: তোমার BazarDor website-এর database-এ থাকতে পারে:

- Product names — আলু, পেঁয়াজ, চাল
- Product prices — ৬০ টাকা, ৮০ টাকা

Database-এর মাধ্যমে আমরা CRUD operation করতে পারি:

- Create: নতুন data সংরক্ষণ
- Read: data দেখা
- Update: data পরিবর্তন
- Delete: data মুছে ফেলা

8. What is SQL?

SQL (Structured Query Language) is a language used to communicate with relational databases.

সহজভাবে, SQL হলো database-এর data তৈরি, পড়া, পরিবর্তন এবং মুছে ফেলার জন্য ব্যবহৃত language।

SQL ব্যবহার করে আমরা database-কে নির্দেশ দিতে পারি কোন data দরকার বা কী পরিবর্তন করতে হবে।

Example:

SELECT * FROM products;

এর অর্থ: `products` table-এর সব data দেখাও।

9. What is NoSQL?

NoSQL refers to a category of databases that do not rely exclusively on the traditional relational table model.

সহজভাবে, NoSQL database-এ data শুধু traditional rows-and-columns table-এ রাখতেই হবে, এমন নিয়ম নেই। Database-এর ধরন অনুযায়ী data documents, key-value pairs, graphs বা wide-column structures-এ রাখা যায়।

Example: একটি product-কে JSON-এর মতো document হিসেবে রাখা যায়।

```
{
  "name": "আলু",
  "price": 60,
  "category": "সবজি",
  "markets": ["কারওয়ান বাজার", "মিরপুর বাজার"]
}
```

এখানে একটি document-এর মধ্যে product-এর বিভিন্ন তথ্য রাখা হয়েছে। NoSQL-এর document databases-এ এ ধরনের structure প্রচলিত।

Difference between SQL and NoSQL

বিষয়	SQL	NoSQL
Database type	Relational database	Non-relational database models
Data structure	Tables, rows ও columns	Documents, key-value, graph ইত্যাদি
Schema	সাধারণত আগে থেকে নির্ধারিত	অনেক ক্ষেত্রে flexible
Relationships	Related tables-এর মধ্যে relationship তৈরি করা সহজ	অনেক model-এ related data একই document-এ রাখা যায়
Queries	Complex queries ও joins-এর জন্য উপযোগী	নির্দিষ্ট data model অনুযায়ী query করা হয়
Scalability	Vertical ও horizontal—দুইভাবেই scale করা যায়	অনেক database horizontal scaling-এর জন্য তৈরি
Examples	PostgreSQL, MySQL	MongoDB, Redis

Important: SQL সবসময় slow বা NoSQL সবসময় fast — এই ধারণা ভুল। Performance নির্ভর করে data structure, query, indexing, workload এবং database design-এর ওপর।

10. When should we use SQL?

SQL database বেছে নাও যখন data-এর মধ্যে clear relationships আছে এবং data consistency খুব গুরুত্বপূর্ণ।

Typical use cases:

- Banking systems ও financial transactions
- E-commerce orders, payments ও inventory
- University management systems
- Employee ও payroll management
- এমন application যেখানে একাধিক table-এর data একসঙ্গে query করতে হয়

Example: একটি e-commerce website-এ `users`, `orders`, `products` ও `payments` আলাদা table-এ রাখা যেতে পারে। SQL দিয়ে তাদের relationship ব্যবহার করে query করা যায়।

11. When should we use NoSQL?

NoSQL database বেছে নাও যখন তোমার application-এর data model এবং workload সেই database-এর সুবিধাগুলো থেকে উপকৃত হয়।

Typical use cases:

- Flexible structure-এর product catalog
- বিভিন্ন ধরনের content-সহ document-based applications
- Large-scale event বা activity data
- Low-latency caching (Redis-এর মতো database)
- নির্দিষ্ট workload-এ অনেক server জুড়ে data distribute করা

Example: একটি content platform-এ বিভিন্ন article-এর fields আলাদা হতে পারে। MongoDB-এর মতো document database-এ সেই documents flexible structure-এ রাখা যায়।

তবে flexible schema দরকার হলেই NoSQL নিতে হবে, এমন নয়। PostgreSQL-এর মতো SQL database-ও JSON data support করে।

SQL vs NoSQL — decision guide

তোমার **requirement**	সম্ভাব্য পছন্দ
Related data ও complex joins	SQL
Transactions ও strong consistency	SQL সাধারণত ভালো starting point
Flexible document structure	NoSQL document database
In-memory caching	Redis-এর মতো key-value database
Highly distributed, specific large-scale workload	উপযুক্ত NoSQL database বিবেচনা করো

এগুলো সাধারণ guideline, কঠোর নিয়ম নয়। SQL ও NoSQL—দুই ধরনের database-ই অনেক ধরনের application তৈরি করতে পারে। Project অনুযায়ী database বেছে নেওয়া শেখো; শুধু জনপ্রিয় বলে কোনো database বেছে নিও না।

12. What is Express.js?

Express.js is a web framework for Node.js that makes it easier to build web servers and APIs.

সহজভাবে, Express.js হলো Node.js-এর ওপর তৈরি একটি framework, যা দিয়ে কম code লিখে backend server এবং API তৈরি করা যায়।

13. Why do we use Express.js?

ধরো, তুমি একটি API তৈরি করতে চাও, যেটি `/api/products` request পেলে সব product return করবে।

Node.js দিয়ে নিজেই request handling, routing এবং response-এর logic লিখতে পারো। কিন্তু Express.js ব্যবহার করলে এই কাজগুলো আরও সহজ ও organized হয়।

Express.js আমাদের সাহায্য করে:

- Routing: কোন URL-এ request এলে কী কাজ হবে, তা নির্ধারণ করতে।
- Request handling: Client থেকে আসা request-এর data পড়তে।
- Response handling: Client-কে JSON বা অন্য response পাঠাতে।
- Middleware: Request ও response-এর মাঝখানে authentication, logging, validation ইত্যাদি কাজ করতে।

Node.js - JavaScript runtime environment

Browser-এর বাইরে JavaScript চালায় এবং server ও file system-এর মতো কাজের জন্য built-in capabilities দেয়।

Express.js - Node.js web framework

Node.js-এর ওপর কাজ করে এবং routing, middleware ও request-response handling সহজ করে।